Bài thực hành 10

Tổng quan

1/ Chủ đề: Smart Home Mini

Xây dựng ứng dụng điều khiển một ngôi nhà thông minh với ba loại thiết bị:

- `SmartLight`;
- `SmartFan`;
- `SmartDoor`.

Hệ thống có các yêu cầu:

1. Các thiết bị có thể bật và tắt. Riêng cửa có thể khóa và mở khóa.
2. Chương trình tạo thiết bị dựa trên loại do người dùng lựa chọn.
3. Cấu hình chung của ngôi nhà chỉ có một thể hiện được sử dụng trong ứng dụng.
4. Người dùng cần kích hoạt các chế độ như **Đi làm** và **Đi ngủ** mà không phải gọi từng thiết bị riêng lẻ.
5. Thiết kế phải cho phép client làm việc chủ yếu qua abstraction thay vì phụ thuộc trực tiếp vào mọi lớp cụ thể.

Các Pattern dự kiến sử dụng

Vấn đề	Thành phần	Pattern
Tách việc tạo thiết bị khỏi client	DeviceFactory	Factory
Chia sẻ một cấu hình ứng dụng	HomeConfiguration	Singleton
Cung cấp thao tác mức cao cho nhiều thiết bị	SmartHomeFacade	Facade

Singleton: Dependency Injection thường giúp quản lý vòng đời đối tượng và kiểm thử tốt hơn.

2/ Sản phẩm

1. Sơ đồ lớp UML.
2. Mã nguồn chạy được.
3. Kết quả kiểm thử tối thiểu cho ba pattern.
4. README.md ngắn, nêu:
    - vị trí sử dụng từng pattern;
    - lợi ích đạt được;
    - một hạn chế của thiết kế.

Phần A: Phân tích thiết kế

Yêu cầu

xác định:

- abstraction chung của thiết bị;
- hành vi riêng của cửa;
- thành phần tạo thiết bị;
- dữ liệu nào thực sự thuộc cấu hình chung;
- các phụ thuộc mà facade cần giữ;
- cách client gọi các chế độ mà không biết chi tiết từng thiết bị.

UML – sơ đồ class:

Phần B: Xây dựng mô hình thiết bị

Mã lập trình khung ví dụ

```
public interface IDevice
{
    void TurnOn();
    void TurnOff();
}

public interface ILockable
{
    void Lock();
    void Unlock();
}
```

Cài đặt:

```
SmartLight : IDevice
SmartFan   : IDevice
SmartDoor  : IDevice, ILockable
```

Yêu cầu

- Mỗi thiết bị có tên riêng được truyền qua constructor.
- Mỗi phương thức in ra trạng thái đủ để quan sát khi chạy chương trình.
- Không đưa `Lock()` vào `IDevice`, vì đèn và quạt không có khả năng khóa.

- Không dùng một lớp cơ sở chỉ để tái sử dụng vài dòng Console.WriteLine.

Ví dụ kết quả

```
Living room light: ON
Living room fan: OFF
Main door: LOCKED
```

Phần C: Mẫu Factory

Yêu cầu sử dụng

```
IDevice light = DeviceFactory.Create("light", "Living room light");
IDevice fan = DeviceFactory.Create("fan", "Living room fan");
IDevice door = DeviceFactory.Create("door", "Main door");
```

Yêu cầu thiết kế

1. Client không trực tiếp gọi constructor của từng loại thiết bị.
2. Factory trả về IDevice.
3. Chuỗi đầu vào không phân biệt chữ hoa và chữ thường.
4. Loại thiết bị không hợp lệ phải gây ra ArgumentException, không trả về null.

Mã ví dụ

```
public static class DeviceFactory
{
    public static IDevice Create(string type, string name)
    {
        return type.Trim().ToLowerInvariant() switch
        {
            "light" => new SmartLight(name),
            "fan"   => new SmartFan(name),
            "door"  => new SmartDoor(name),
            _ => throw new ArgumentException(
                $"Unsupported device type: {type}")
        };
    }
}
```

Câu hỏi để phân tích

1. Factory đã che giấu điều gì đối với client?

2. Khi thêm SmartCamera, những thành phần nào cần sửa?

Phần D: Singleton

Yêu cầu sử dụng

```
HomeConfiguration config1 = HomeConfiguration.Instance;
HomeConfiguration config2 = HomeConfiguration.Instance;

Console.WriteLine(ReferenceEquals(config1, config2)); // True
```

Mã tham khảo an toàn luồng với `Lazy<T>`

```
public sealed class HomeConfiguration
{
    private static readonly Lazy<HomeConfiguration> lazy =
        new(() => new HomeConfiguration());

    public static HomeConfiguration Instance => lazy.Value;

    public string HomeName { get; set; } = "My Home";
    public int NightLightLevel { get; set; } = 30;

    private HomeConfiguration()
    {
    }
}
```

Kiểm tra bắt buộc

```
var c1 = HomeConfiguration.Instance;
var c2 = HomeConfiguration.Instance;

if (!ReferenceEquals(c1, c2))
{
    throw new Exception("Singleton implementation is incorrect.");
}
```

Câu hỏi phản biện

1. Constructor phải có mức truy cập nào?

2. Vì sao lớp được đánh dấu `sealed`?
3. Singleton gây khó khăn gì cho unit test?
4. Nếu dùng .NET Dependency Injection với lifetime `AddSingleton`, có còn nhất thiết phải tự cài Singleton pattern không?

Phần E: Facade

Yêu cầu

`SmartHomeFacade` cung cấp hai thao tác mức cao:

`GoToWork()`

- Tắt đèn.
- Tắt quạt.
- Khóa cửa.

`GoToSleep()`

- Tắt quạt.
- Khóa cửa.
- Bật đèn ngủ.
- Đọc mức sáng ban đêm từ `HomeConfiguration` và thông báo mức sáng được áp dụng.

Cách sử dụng mong đợi

```
var facade = new SmartHomeFacade(light, fan, (SmartDoor)door);
facade.GoToWork();
facade.GoToSleep();
```

Mã tham khảo

```
public sealed class SmartHomeFacade
{
    private readonly IDevice light;
    private readonly IDevice fan;
    private readonly ILockable door;

    public SmartHomeFacade(
        IDevice light,
        IDevice fan,
        ILockable door)
```

```
    {
        this.light = light;
        this.fan = fan;
        this.door = door;
    }

    public void GoToWork()
    {
        light.TurnOff();
        fan.TurnOff();
        door.Lock();
    }

    public void GoToSleep()
    {
        fan.TurnOff();
        door.Lock();
        light.TurnOn();

        int level = HomeConfiguration.Instance.NightLightLevel;
        Console.WriteLine($"Night light level: {level}%");
    }
}
```

Chú ý:

- Facade không thay thế các lớp thiết bị.
- Client vẫn có thể điều khiển riêng từng thiết bị khi cần.
- Facade chỉ cung cấp giao diện đơn giản hơn cho một nhóm thao tác thường dùng.
-

Phần F: Kiểm thử và cải tiến

Sinh viên thực hiện tối thiểu các kiểm tra sau:

Mã	Tình huống kiểm tra	Kết quả mong đợi
T1	Tạo `light`, `fan`, `door`	Factory trả đúng kiểu đối tượng
T2	Tạo loại `camera` khi chưa hỗ trợ	Phát sinh `ArgumentException`
T3	Lấy `HomeConfiguration.Instance` hai lần	Hai biến cùng tham chiếu một đối tượng
T4	Gọi `GoToWork()`	Đèn tắt, quạt tắt, cửa khóa
T5	Gọi `GoToSleep()`	Quạt tắt, cửa khóa, đèn bật, hiện mức sáng

Phần G: Câu hỏi trả lời

Trả lời ngắn trong README.md:

1. Factory đang che giấu quyết định thiết kế nào?
2. Singleton bảo đảm điều gì, nhưng tạo ra rủi ro nào?
3. Facade khác Factory ở mục đích chính như thế nào?
4. Nếu bỏ SmartHomeFacade, client sẽ phải biết thêm những chi tiết nào?